
TRÍ TUỆ NHÂN TẠO

Machine Learning cơ bản

Chương: Gradient Descent

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
Mục tiêu và ký hiệu	4
1 Chương 1. Cơ sở Gradient Descent	5
1.1 1.1. Giới thiệu	5
1.1.1 Gradient Descent	6
1.2 1.2. Gradient Descent cho hàm một biến	7
1.2.1 Ví dụ đơn giản với Python	7
1.3 1.3. Gradient Descent cho hàm nhiều biến	10
1.3.1 Ứng dụng: Linear Regression	10
1.3.2 Ví dụ trên Python và lưu ý khi lập trình	11
1.3.3 Đường đồng mức (level sets)	15
1.4 1.4. Ví dụ: hai cực tiểu phụ thuộc khởi tạo	16
1.5 1.5. Kết chương 1	17
1.6 Tài liệu tham khảo	17
2 Chương 2. Biến thể và kỹ thuật nâng cao	17
2.1 1. Các thuật toán tối ưu Gradient Descent	17
2.1.1 1.1. Momentum	17
2.1.2 1.2. Nesterov accelerated gradient (NAG)	20
2.1.3 1.3. Các thuật toán khác	22
2.2 2. Biến thể của Gradient Descent	22
2.2.1 2.1. Batch Gradient Descent	23
2.2.2 2.2. Stochastic Gradient Descent	23
2.2.3 2.3. Mini-batch Gradient Descent	26
2.3 3. Stopping Criteria (điều kiện dừng)	27
2.4 4. Một phương pháp tối ưu đơn giản khác: Newton's method	28
2.4.1 Newton's method cho giải phương trình $f(x) = 0$	28
2.4.2 Newton's method trong bài toán tìm local minimum	29
2.4.3 Hạn chế của Newton's method	29
2.5 5. Kết luận	29
2.6 6. Tài liệu tham khảo	29
Tổng hợp và củng cố	30
2.7 Bảng so sánh các biến thể GD	30
2.8 Bổ trợ: các optimizer phổ biến trong Deep Learning	30
2.9 Câu hỏi củng cố	31

Lời nói đầu

Vị trí trong giáo trình

Chương trình bày thuật toán **Gradient Descent (GD)** và các biến thể — nền tảng để huấn luyện hầu hết mô hình học máy có hàm mất mát khả vi. Nội dung cốt lõi biên soạn theo chuỗi bài Machine Learning cơ bản (Vũ Hồng Nghi); được chỉnh lý theo phong cách **bài giảng / giáo trình** (không giữ văn phong blog), kèm phần hỗ trợ giải thích bản chất.

Tài liệu tham chiếu gốc

- Bài 7 — Gradient Descent (phần 1/2): <https://machinelearningcoban.com/2017/01/12/gradientdescent/>
- Bài 8 — Gradient Descent (phần 2/2): <https://machinelearningcoban.com/2017/01/16/gradientdescent2/>

Cách học chương này

- Đọc **Mục tiêu và ký hiệu**, sau đó Chương 1 (lý thuyết + code), rồi Chương 2 (SGD, momentum, Newton).
- Chạy lại các đoạn Python; thử đổi η , x_0 để quan sát hội tụ.
- Kết thúc bằng **Tổng hợp và củng cố** (bảng so sánh, câu hỏi ôn tập).

Mục tiêu và ký hiệu

Mục tiêu học tập

Sau chương này, người học có thể:

1. Giải thích **bản chất** của Gradient Descent: đi theo hướng **ngược gradient** để giảm hàm mục tiêu (thường là hàm mất mát).
2. Viết và chạy GD cho hàm một biến và nhiều biến; kiểm tra gradient bằng **numerical gradient**.
3. Nhận biết ảnh hưởng của **điểm khởi tạo**, **learning rate** và hình dạng đường đồng mức tới hội tụ.
4. Phân biệt **Batch GD**, **SGD**, **mini-batch**; mô tả vai trò của **Momentum**, **NAG** và ý tưởng **Newton**.
5. Chọn điều kiện dừng phù hợp trong thực nghiệm.

Kiến thức tiên quyết

Đạo hàm hàm một biến và nhiều biến; ma trận và nhân ma trận; Linear Regression (hàm mất mát dạng bình phương tối thiểu). Phần code dùng **numpy** và **matplotlib**.

Ký hiệu thống nhất

- $f, \mathcal{L}, J$: hàm mục tiêu / hàm mất mát.
- θ hoặc $\mathbf{w}$: vector tham số cần tối ưu.
- η : learning rate (bước học); γ : hệ số momentum.
- $\nabla f, \nabla_{\theta} \mathcal{L}$: gradient (vector chỉ hướng **tăng nhanh nhất** của hàm).
- x^*, θ^* : điểm cực tiểu (local hoặc global tùy bài toán).
- GD: Gradient Descent; SGD: Stochastic Gradient Descent.

Bản chất Gradient Descent — đọc trước khi vào chi tiết

Gradient Descent không phải công thức đóng cho nghiệm tối ưu, mà là quy trình **lặp**:

1. Tại tham số hiện tại, tính **gradient** của hàm mất mát.
2. Cập nhật tham số một bước nhỏ theo hướng **trừ gradient** (vì gradient chỉ hướng **đi lên dốc nhanh nhất** của hàm).
3. Dừng khi gradient đủ nhỏ hoặc đạt giới hạn vòng lặp.

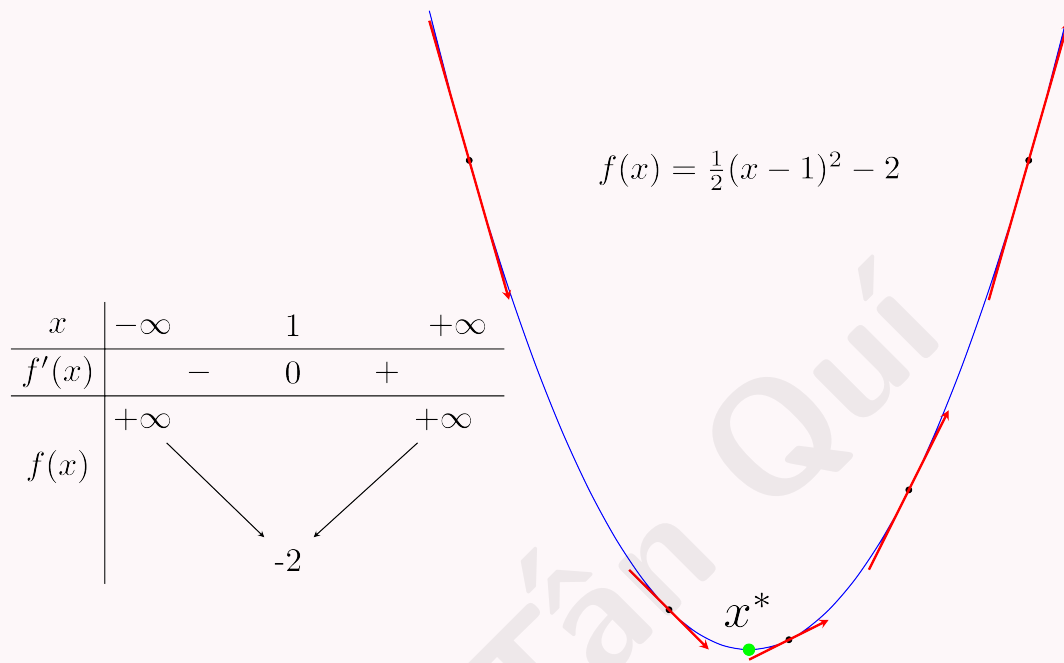
Trong Machine Learning, ta thường **không giải được** $\nabla \mathcal{L} = 0$ bằng đại số (hàm phi tuyến, số chiều lớn, dữ liệu lớn). GD thay thế bằng đi từng bước nhỏ — giống “lăn xuống thung lũng” trên bề mặt lỗi (loss landscape).

1 Chương 1. Cơ sở Gradient Descent

1.1 1.1. Giới thiệu

Bề mặt hàm số và cực tiểu

Hình minh họa dưới đây thể hiện đồ thị hàm một biến và vị trí cực tiểu:



Điểm màu xanh lục là điểm **local minimum** (cực tiểu), và cũng là điểm làm cho hàm số đạt giá trị nhỏ nhất. Từ đây trở đi, ta dùng *local minimum* để thay cho điểm cực tiểu, *global minimum* để thay cho điểm mà tại đó hàm số đạt giá trị nhỏ nhất. **Global minimum** là một trường hợp đặc biệt của local minimum.

Local minimum và đạo hàm

Giả sử chúng ta đang quan tâm đến một hàm số một biến có đạo hàm mọi nơi. Nhắc lại vài điều đã quá quen thuộc:

Điểm local minimum x^* của hàm số là điểm có đạo hàm $f'(x^*)$ bằng 0. Hơn thế nữa, trong lân cận của nó, đạo hàm của các điểm phía bên trái x^* là không dương, đạo hàm của các điểm phía bên phải x^* là không âm.

Đường tiếp tuyến với đồ thị hàm số đó tại một điểm bất kỳ có hệ số góc chính bằng đạo hàm của hàm số tại điểm đó.

Dấu đạo hàm quanh cực tiểu

Trong hình phía trên, các điểm bên trái của điểm local minimum màu xanh lục có đạo hàm âm, các điểm bên phải có đạo hàm dương. Và đối với hàm số này, càng xa về phía trái của điểm local minimum thì đạo hàm càng âm, càng xa về phía phải thì đạo hàm càng dương.

1.1.1 Gradient Descent

Tại sao cần tối ưu trong Machine Learning?

Trong Machine Learning nói riêng và Toán Tối Ưu nói chung, chúng ta thường xuyên phải tìm giá trị nhỏ nhất (hoặc đôi khi là lớn nhất) của một hàm số nào đó. Ví dụ như các hàm mất mát trong hai bài Linear Regression và K-means Clustering. Nhìn chung, việc tìm global minimum của các hàm mất mát trong Machine Learning là rất phức tạp, thậm chí là bất khả thi. Thay vào đó, người ta thường cố gắng tìm các điểm local minimum, và ở một mức độ nào đó, coi đó là nghiệm cần tìm của bài toán.

Từ phương trình đạo hàm bằng 0 đến lặp

Các điểm local minimum là nghiệm của phương trình đạo hàm bằng 0. Nếu bằng một cách nào đó có thể tìm được toàn bộ (hữu hạn) các điểm cực tiểu, ta chỉ cần thay từng điểm local minimum đó vào hàm số rồi tìm điểm làm cho hàm có giá trị nhỏ nhất (bước so sánh giá trị hàm tại các cực tiểu). Tuy nhiên, trong hầu hết các trường hợp, việc giải phương trình đạo hàm bằng 0 là bất khả thi. Nguyên nhân có thể đến từ sự phức tạp của dạng của đạo hàm, từ việc các điểm dữ liệu có số chiều lớn, hoặc từ việc có quá nhiều điểm dữ liệu.

Hướng tiếp cận phổ biến nhất là xuất phát từ một điểm mà chúng ta coi là gần với nghiệm của bài toán, sau đó dùng một phép toán lặp để tiến dần đến điểm cần tìm, tức đến khi đạo hàm gần với 0. **Gradient Descent** (viết gọn là GD) và các biến thể của nó là một trong những phương pháp được dùng nhiều nhất.

Cấu trúc chương

Nội dung GD được chia hai chương: **Chương 1** (mục này) — ý tưởng, cập nhật một/nhiều biến, Linear Regression và kiểm tra gradient; **Chương 2** — momentum, NAG, Batch/SGD/mini-batch, điều kiện dừng, Newton. Bài toán *large-scale* (số chiều và số mẫu lớn) là động lực cho các biến thể ở Chương 2.

Thuật toán Gradient Descent (khung chung)

Đầu vào: hàm khả vi $f(\theta)$ (hoặc $\mathcal{L}$), learning rate $\eta > 0$, điểm khởi tạo θ_0 , tiêu chí dừng.

Lặp cho $t = 0, 1, 2, \dots$ đến khi dừng:

1. Tính $\mathbf{g}_t = \nabla_{\theta} f(\theta_t)$.
2. Cập nhật $\theta_{t+1} = \theta_t - \eta \mathbf{g}_t$.
3. Nếu $\|\mathbf{g}_t\|$ nhỏ hơn ngưỡng (hoặc đạt số vòng lặp tối đa) thì dừng.

Đầu ra: θ_t xấp xỉ cực tiểu (thường là local minimum).

1.2 Gradient Descent cho hàm một biến

Quan sát từ đồ thị

Quay trở lại hình vẽ ban đầu và các quan sát đã nêu. Giả sử x_t là điểm ta tìm được sau vòng lặp thứ t . Ta cần tìm một thuật toán để đưa x_t về càng gần x^* càng tốt.

Trong hình đầu tiên, chúng ta lại có thêm hai quan sát nữa:

Nếu đạo hàm của hàm số tại x_t : $f'(x_t) > 0$ thì x_t nằm về bên phải so với x^* (và ngược lại). Để điểm tiếp theo x_{t+1} gần với x^* hơn, chúng ta cần di chuyển x_t về phía bên trái, tức về phía âm. Nói cách khác, chúng ta cần di chuyển ngược dấu với đạo hàm:

$$x_{t+1} = x_t + \Delta$$

Trong đó Δ là một đại lượng ngược dấu với đạo hàm $f'(x_t)$.

x_t càng xa x^* về phía bên phải thì $f'(x_t)$ càng lớn hơn 0 (và ngược lại). Vậy, lượng di chuyển Δ , một cách trực quan nhất, là tỉ lệ thuận với $-f'(x_t)$.

Cập nhật Gradient Descent một biến

Hai nhận xét phía trên cho ta công thức cập nhật:

$$x_{t+1} = x_t - \eta f'(x_t)$$

Trong đó η (đọc là *eta*) là **learning rate** (bước học), số dương. Dấu **trừ** vì ta đi **ngược** hướng đạo hàm — gradient chỉ hướng **tăng** nhanh nhất của f , còn ta cần **giảm** f (Gradient Descent = “đi xuống”).

Điều kiện giảm hàm (một biến, η nhỏ)

Với f khả vi, nếu $f'(x_t) \neq 0$ và $|\eta f'(x_t)|$ đủ nhỏ thì $f(x_{t+1}) < f(x_t)$ (xấp xỉ bậc nhất). Đây là cơ sở lý thuyết cho việc chọn η không quá lớn.

1.2.1 Ví dụ đơn giản với Python

Hàm $f(x) = x^2 + 5 \sin(x)$

Xét hàm số $f(x) = x^2 + 5 \sin(x)$ với đạo hàm $f'(x) = 2x + 5 \cos(x)$ (hàm này được chọn vì nó không dễ tìm nghiệm của đạo hàm bằng 0 như hàm phía trên). Giả sử bắt đầu từ một điểm x_0 nào đó, tại vòng lặp thứ t , chúng ta sẽ cập nhật như sau:

$$x_{t+1} = x_t - \eta(2x_t + 5 \cos(x_t))$$

Đầu tiên, ta khai báo vài thư viện quen thuộc:

```
# To support both python 2 and python 3
from __future__ import division, print_function, unicode_literals
import math
import numpy as np
import matplotlib.pyplot as plt
```

Tiếp theo, ta định nghĩa các hàm số:

1. `grad` để tính đạo hàm;
2. `cost` để tính giá trị của hàm số. Hàm này không sử dụng trong thuật toán nhưng thường được dùng để kiểm tra việc tính đạo hàm đúng không hoặc để xem giá trị của hàm số có giảm theo mỗi vòng lặp hay không;
3. `myGD1` là phần chính thực hiện thuật toán Gradient Descent nêu phía trên. Đầu vào của hàm số này là learning rate và điểm bắt đầu. Thuật toán dừng lại khi đạo hàm có độ lớn đủ nhỏ.

```
def grad(x):
    return 2*x+ 5*np.cos(x)

def cost(x):
    return x**2 + 5*np.sin(x)

def myGD1(eta, x0):
    x = [x0]
    for it in range(100):
        x_new = x[-1] - eta*grad(x[-1])
        if abs(grad(x_new)) < 1e-3:
            break
        x.append(x_new)
    return (x, it)
```

Điểm khởi tạo khác nhau.

Thử $x_0 = -5$ và $x_0 = 5$

Sau khi có các hàm cần thiết, thử nghiệm với nghiệm với các điểm khởi tạo khác nhau là $x_0 = -5$ và $x_0 = 5$:

```
(x1, it1) = myGD1(.1, -5)
(x2, it2) = myGD1(.1, 5)
print('Solution x1 = %f, cost = %f, obtained after %d iterations'%(x1[-1],
    cost(x1[-1]), it1))
print('Solution x2 = %f, cost = %f, obtained after %d iterations'%(x2[-1],
    cost(x2[-1]), it2))
```

Kết quả in ra:

```
Solution x1 = -1.110667, cost = -3.246394, obtained after 11 iterations
Solution x2 = -1.110341, cost = -3.246394, obtained after 29 iterations
```

Vậy là với các điểm ban đầu khác nhau, thuật toán của chúng ta tìm được nghiệm gần giống nhau, mặc dù với tốc độ hội tụ khác nhau. Dưới đây là hình ảnh minh họa thuật toán GD cho bài toán này .

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img03.png. Chạy download_mlc_b_images.py.

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img04.png. Chạy download_mlcb_images.py.

So sánh hai điểm khởi tạo

Từ hình minh họa trên ta thấy rằng ở hình bên trái, tương ứng với $x_0 = -5$, nghiệm hội tụ nhanh hơn, vì điểm ban đầu x_0 gần với nghiệm $x^* \approx -1$ hơn. Hơn nữa, với $x_0 = 5$ ở hình bên phải, đường đi của nghiệm có chứa một khu vực có đạo hàm khá nhỏ gần điểm có hoành độ bằng 2. Điều này khiến cho thuật toán la cà ở đây khá lâu. Khi vượt qua được điểm này thì mọi việc diễn ra rất tốt đẹp.

Learning rate khác nhau.**Cùng $x_0 = -5$, η khác nhau**

Tốc độ hội tụ của GD không những phụ thuộc vào điểm khởi tạo ban đầu mà còn phụ thuộc vào learning rate. Dưới đây là một ví dụ với cùng điểm khởi tạo $x_0 = -5$ nhưng learning rate khác nhau:

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img05.png. Chạy download_mlcb_images.py.

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img06.png. Chạy download_mlcb_images.py.

Hai trường hợp learning rate

Ta quan sát thấy hai điều:

1. Với learning rate nhỏ $\eta = 0.01$, tốc độ hội tụ rất chậm. Trong ví dụ này giới hạn tối đa 100 vòng lặp nên thuật toán dừng lại trước khi tới đích, mặc dù đã rất gần. Trong thực tế, khi việc tính toán trở nên phức tạp, learning rate quá thấp sẽ ảnh hưởng tới tốc độ của thuật toán rất nhiều, thậm chí không bao giờ tới được đích.
2. Với learning rate lớn $\eta = 0.5$, thuật toán tiến rất nhanh tới gần đích sau vài vòng lặp. Tuy nhiên, thuật toán không hội tụ được vì bước nhảy quá lớn, khiến nó cứ quần quanh ở đích.

Chọn learning rate

Việc lựa chọn learning rate rất quan trọng trong các bài toán thực tế. Việc lựa chọn giá trị này phụ thuộc nhiều vào từng bài toán và phải làm một vài thí nghiệm để chọn ra giá trị tốt nhất. Ngoài ra, tùy vào một số bài toán, GD có thể làm việc hiệu quả hơn bằng cách chọn ra learning rate phù hợp hoặc chọn learning rate khác nhau ở mỗi vòng lặp. Phần sau trình bày vấn đề này ở phần 2.

1.3 Gradient Descent cho hàm nhiều biến**Cập nhật với vector tham số θ**

Giả sử ta cần tìm global minimum cho hàm $f(\theta)$ trong đó θ (*theta*) là một vector, thường được dùng để ký hiệu tập hợp các tham số của một mô hình cần tối ưu (trong Linear Regression thì các tham số chính là hệ số $\mathbf{w}$). Đạo hàm của hàm số đó tại một điểm θ bất kỳ được ký hiệu là $\nabla_{\theta} f(\theta)$ (hình tam giác ngược đọc là *nabla*). Tương tự như hàm một biến, thuật toán GD cho hàm nhiều biến cũng bắt đầu bằng một điểm dự đoán θ_0 , sau đó, ở vòng lặp thứ t , quy tắc cập nhật là:

Quy tắc cập nhật nhiều biến

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

Hoặc viết dưới dạng đơn giản hơn: $\theta \leftarrow \theta - \eta \nabla_{\theta} f(\theta)$.

Quy tắc cần nhớ: luôn luôn đi ngược hướng với đạo hàm.

Việc tính toán đạo hàm của các hàm nhiều biến là một kỹ năng cần thiết.

1.3.1 Ứng dụng: Linear Regression**Tối ưu hàm mất mát Linear Regression**

Áp dụng GD cho bài toán Linear Regression: tối ưu hàm mất mát thay vì (hoặc để đổi chiều với) nghiệm đóng $\mathbf{w} = (\bar{\mathbf{X}}^T \bar{\mathbf{X}})^{-1} \bar{\mathbf{X}}^T \mathbf{y}$.

GD so với nghiệm công thức

Với MSE và ma trận đầy đủ hạng, nghiệp vụ tối thiểu là duy nhất (hàm lồi). GD và công thức ma trận **cùng hướng tới cùng $\mathbf{w}^*$** nếu η phù hợp. GD vẫn quan trọng vì: (i) không cần lưu nghịch đảo ma trận lớn; (ii) mở rộng được khi thêm regularization hoặc mô hình phi tuyến; (iii) là mẫu cho SGD khi N rất lớn.

Hàm mất mát và gradient

Hàm mất mát của Linear Regression là:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \bar{\mathbf{X}}\mathbf{w}\|_2^2$$

Đạo hàm của hàm mất mát là:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{N} \bar{\mathbf{X}}^T (\bar{\mathbf{X}}\mathbf{w} - \mathbf{y}) \quad (1)$$

Hàm mất mát có chia cho N

Hàm này có khác một chút so với hàm trong bài Linear Regression (không chia N). Mẫu số có thêm N là số lượng dữ liệu trong training set. Việc lấy trung bình cộng của lỗi này nhằm giúp tránh trường hợp hàm mất mát và đạo hàm có giá trị là một số rất lớn, ảnh hưởng tới độ chính xác của các phép toán khi thực hiện trên máy tính. Về mặt toán học, nghiệm của hai bài toán là như nhau.

1.3.2 Ví dụ trên Python và lưu ý khi lập trình

Load thư viện và tạo dữ liệu

Load thư viện:

```
# To support both python 2 and python 3
from __future__ import division, print_function, unicode_literals
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
np.random.seed(2)
```

Tiếp theo, chúng ta tạo 1000 điểm dữ liệu được chọn gần với đường thẳng $y = 4 + 3x$, hiển thị chúng và tìm nghiệm theo công thức:

```
X = np.random.rand(1000, 1)
y = 4 + 3 * X + .2*np.random.randn(1000, 1) # noise added

# Building Xbar
one = np.ones((X.shape[0],1))
Xbar = np.concatenate((one, X), axis = 1)

A = np.dot(Xbar.T, Xbar)
b = np.dot(Xbar.T, y)
w_lr = np.dot(np.linalg.pinv(A), b)
print('Solution found by formula: w = ',w_lr.T)

# Display result
w = w_lr
w_0 = w[0][0]
w_1 = w[1][0]
x0 = np.linspace(0, 1, 2, endpoint=True)
y0 = w_0 + w_1*x0

# Draw the fitting line
plt.plot(X.T, y.T, 'b.') # data
plt.plot(x0, y0, 'y', linewidth = 2) # the fitting line
```

```
plt.axis([0, 1, 0, 10])
plt.show()
```

Kết quả:

```
Solution found by formula: w = [[ 4.00305242  2.99862665]]
```

Đường thẳng tìm được là đường có màu vàng có phương trình $y \approx 4 + 2.998x$.

Tiếp theo ta viết đạo hàm và hàm mất mát:

```
def grad(w):
    N = Xbar.shape[0]
    return 1/N * Xbar.T.dot(Xbar.dot(w) - y)

def cost(w):
    N = Xbar.shape[0]
    return .5/N*np.linalg.norm(y - Xbar.dot(w), 2)**2;
```

Kiểm tra đạo hàm.

Numerical gradient

Việc tính đạo hàm của hàm nhiều biến thông thường khá phức tạp và rất dễ mắc lỗi, nếu chúng ta tính sai đạo hàm thì thuật toán GD không thể chạy đúng được. Trong thực nghiệm, có một cách để kiểm tra liệu đạo hàm tính được có chính xác không. Cách này dựa trên định nghĩa của đạo hàm (cho hàm một biến):

$$f'(x) = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon) - f(x)}{\varepsilon}$$

Một cách thường được sử dụng là lấy một giá trị ε rất nhỏ, ví dụ 10^{-6} , và sử dụng công thức xấp xỉ hai phía. Cách tính này được gọi là **numerical gradient**.

Xấp xỉ đạo hàm hai phía

$$f'(x) \approx \frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon} \quad (2)$$

Tại sao xấp xỉ hai phía?

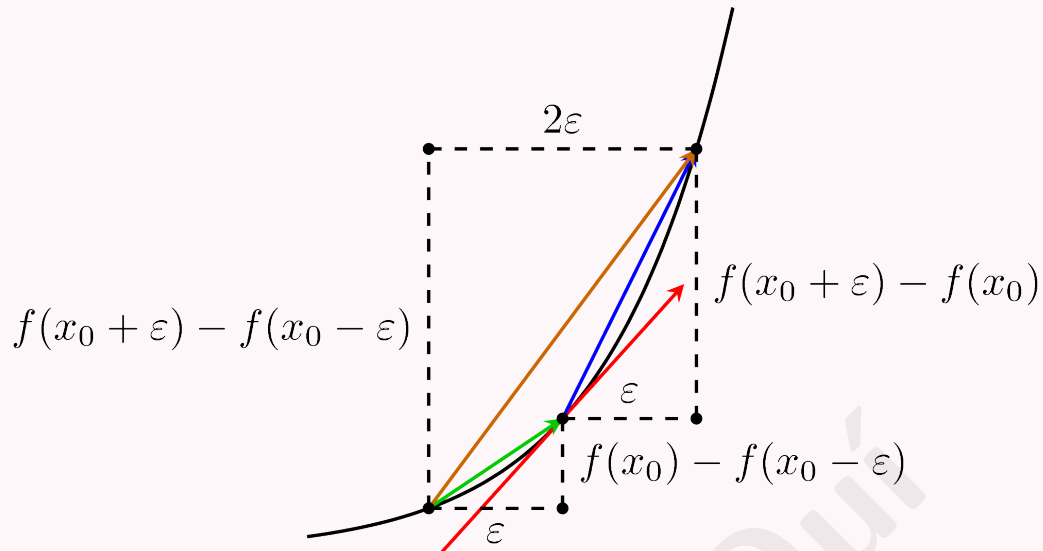
Câu hỏi: Tại sao công thức xấp xỉ hai phía trên đây lại được sử dụng rộng rãi, sao không sử dụng công thức xấp xỉ đạo hàm bên phải hoặc bên trái?

Có hai cách giải thích cho vấn đề này, một bằng hình học, một bằng giải tích.

Giải thích bằng hình học.

Ba vector xấp xỉ so với đạo hàm chính xác

Quan sát hình dưới đây:



Trong hình, vector màu đỏ là đạo hàm chính xác của hàm số tại điểm có hoành độ bằng x_0 . Vector màu xanh lam (có vẻ là hơi tím sau khi convert từ .pdf sang .png) thể hiện cách xấp xỉ đạo hàm phía phải. Vector màu xanh lục thể hiện cách xấp xỉ đạo hàm phía trái. Vector màu nâu thể hiện cách xấp xỉ đạo hàm hai phía. Trong ba vector xấp xỉ đó, vector xấp xỉ hai phía màu nâu là gần với vector đỏ nhất nếu xét theo hướng. Sự khác biệt giữa các cách xấp xỉ còn lớn hơn nữa nếu tại điểm x , hàm số bị bẻ cong mạnh hơn. Khi đó, xấp xỉ trái và phải sẽ khác nhau rất nhiều. Xấp xỉ hai bên sẽ ổn định hơn.

Giải thích bằng giải tích.

Khai triển Taylor và sai số $O(\varepsilon)$ vs. $O(\varepsilon^2)$

Chúng ta cùng quay lại một chút với Giải tích I năm thứ nhất đại học: Khai triển Taylor. Với ε rất nhỏ, ta có hai xấp xỉ sau:

$$f(x + \varepsilon) \approx f(x) + f'(x)\varepsilon + \frac{f''(x)}{2}\varepsilon^2 + \dots$$

và:

$$f(x - \varepsilon) \approx f(x) - f'(x)\varepsilon + \frac{f''(x)}{2}\varepsilon^2 - \dots$$

Từ đó ta có:

$$\frac{f(x + \varepsilon) - f(x)}{\varepsilon} \approx f'(x) + \frac{f''(x)}{2}\varepsilon + \dots = f'(x) + O(\varepsilon) \quad (3)$$

$$\frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon} \approx f'(x) + \frac{f'''(x)}{6}\varepsilon^2 + \dots = f'(x) + O(\varepsilon^2) \quad (4)$$

Từ đó, nếu xấp xỉ đạo hàm bằng công thức (3) (xấp xỉ đạo hàm phải), sai số sẽ là $O(\varepsilon)$. Trong khi đó, nếu xấp xỉ đạo hàm bằng công thức (4) (xấp xỉ đạo hàm hai phía), sai số sẽ là $O(\varepsilon^2) \ll O(\varepsilon)$ nếu ε nhỏ.

Cả hai cách giải thích trên đây đều cho chúng ta thấy rằng, xấp xỉ đạo hàm hai phía là xấp xỉ tốt hơn.

Với hàm nhiều biến.

Áp dụng numerical gradient cho nhiều biến

Với hàm nhiều biến, công thức (2) được áp dụng cho từng biến khi các biến khác cố định. Cách tính này thường cho giá trị khá chính xác. Tuy nhiên, cách này không được sử dụng để tính đạo hàm vì độ phức tạp quá cao so với cách tính trực tiếp. Khi so sánh đạo hàm này với đạo hàm chính xác tính theo công thức, người ta thường giảm số chiều dữ liệu và giảm số điểm dữ liệu để thuận tiện cho tính toán. Một khi đạo hàm tính được rất gần với numerical gradient, chúng ta có thể tự tin rằng đạo hàm tính được là chính xác.

Code kiểm tra đạo hàm

Dưới đây là một đoạn code đơn giản để kiểm tra đạo hàm và có thể áp dụng với một hàm số (của một vector) bất kỳ với `cost` và `grad` đã tính ở phía trên.

```
def numerical_grad(w, cost):
    eps = 1e-4
    g = np.zeros_like(w)
    for i in range(len(w)):
        w_p = w.copy()
        w_n = w.copy()
        w_p[i] += eps
        w_n[i] -= eps
        g[i] = (cost(w_p) - cost(w_n))/(2*eps)
    return g

def check_grad(w, cost, grad):
    w = np.random.rand(w.shape[0], w.shape[1])
    grad1 = grad(w)
    grad2 = numerical_grad(w, cost)
    return True if np.linalg.norm(grad1 - grad2) < 1e-6 else False

print( 'Checking gradient...', check_grad(np.random.rand(2, 1), cost, grad))
```

Kết quả:

```
Checking gradient... True
```

(Với các hàm số khác, người học chỉ cần viết lại hàm `grad` và `cost` ở phần trên rồi áp dụng đoạn code này để kiểm tra đạo hàm. Nếu hàm số là hàm của một ma trận thì chúng ta thay đổi một chút trong hàm `numerical_grad`, có thể chỉnh sửa tương tự).

Với bài toán Linear Regression, cách tính đạo hàm như trong (1) phía trên được coi là đúng vì sai số giữa hai cách tính là rất nhỏ (nhỏ hơn 10^{-6}). Sau khi có được đạo hàm chính xác, chúng ta viết hàm cho GD:

```
def myGD(w_init, grad, eta):
    w = [w_init]
```

```

for it in range(100):
    w_new = w[-1] - eta*grad(w[-1])
    if np.linalg.norm(grad(w_new))/len(w_new) < 1e-3:
        break
    w.append(w_new)
    return (w, it)

w_init = np.array([[2], [1]])
(w1, it1) = myGD(w_init, grad, 1)
print('Solution found by GD: w = ', w1[-1].T, ',\nafter %d iterations.' %(
    it1+1))

```

Kết quả:

```

Solution found by GD: w = [[ 4.01780793  2.97133693]] ,
after 49 iterations.

```

Sau 49 vòng lặp, thuật toán đã hội tụ với một nghiệm khá gần với nghiệm tìm được theo công thức.

Dưới đây là hình động minh họa thuật toán GD.

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img09.png. Chạy download_mlcb_images.py.

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img10.png. Chạy download_mlcb_images.py.

Hai panel minh họa GD cho Linear Regression

Trong hình bên trái, các đường thẳng màu đỏ là nghiệm tìm được sau mỗi vòng lặp. Trong hình bên phải, đây là khái niệm: **đường đồng mức**.

1.3.3 Đường đồng mức (level sets)

Level sets

Với đồ thị của một hàm số với hai biến đầu vào cần được vẽ trong không gian ba chiều, nhiều khi chúng ta khó nhìn được nghiệm có khoảng tọa độ bao nhiêu. Trong toán tối ưu, người ta thường dùng một cách vẽ sử dụng khái niệm **đường đồng mức** (level sets). Trong bản đồ tự nhiên trong các bản đồ tự nhiên, để miêu tả độ cao của các dãy núi, người ta dùng nhiều đường cong kín bao quanh nhau như sau:

Ví dụ về đường đồng mức trong các bản đồ tự nhiên. (Địa lý 6: Đường đồng mức là những đường như thế nào?)

Các vòng nhỏ màu đỏ hơn thể hiện các điểm ở trên cao hơn.

Trong toán tối ưu, người ta cũng dùng phương pháp này để thể hiện các bề mặt trong

không gian hai chiều.

Level sets trong ví dụ Linear Regression

Quay trở lại với hình minh họa thuật toán GD cho bài toán Linear Regression bên trên, hình bên phải là hình biểu diễn các level sets (đường đồng mức). Tức là tại các điểm trên cùng một vòng, hàm mất mát có giá trị như nhau. Trong ví dụ này, hình minh họa giá trị của hàm số tại một số vòng. Các vòng màu xanh có giá trị thấp, các vòng tròn màu đỏ phía ngoài có giá trị cao hơn. Điểm này khác một chút so với đường đồng mức trong tự nhiên là các vòng bên trong thường thể hiện một thung lũng hơn là một đỉnh núi (vì chúng ta đang đi tìm giá trị nhỏ nhất).

Learning rate nhỏ hơn trên Linear Regression

Thử nghiệm với learning rate nhỏ hơn, kết quả như sau:

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img12.png. Chạy download_mlc_b_images.py.

Hội tụ chậm khi η quá nhỏ

Tốc độ hội tụ đã chậm đi nhiều, thậm chí sau 99 vòng lặp, GD vẫn chưa tới gần được nghiệm tốt nhất. Trong các bài toán thực tế, chúng ta cần nhiều vòng lặp hơn 99 rất nhiều, vì số chiều và số điểm dữ liệu thường là rất lớn.

1.4 Ví dụ: hai cực tiểu phụ thuộc khởi tạo

Hàm $f(x, y) = (x^2 + y - 7)^2 + (x - y + 1)^2$

Trước khi chuyển sang Chương 2, xét thêm một ví dụ hàm hai biến:

Hàm số $f(x, y) = (x^2 + y - 7)^2 + (x - y + 1)^2$ có hai điểm local minimum màu xanh lục tại $(2, 3)$ và $(-3, -2)$, và chúng cũng là hai điểm global minimum. Trong ví dụ này, tùy vào điểm khởi tạo mà chúng ta thu được các nghiệm cuối cùng khác nhau.

Thiếu hình minh họa

Không tìm thấy images/01_gradient_descent_part1/img14.png. Chạy download_mlc_b_images.py.

1.5 1.5. Kết chương 1

Tóm tắt Chương 1

Chương này xây dựng **Gradient Descent** từ quan sát đạo hàm một biến đến cập nhật vector θ , với minh họa Python và Linear Regression. Các ý then chốt: đi **ngược gradient**; η và θ_0 quyết định hội tụ; kiểm tra $\nabla \mathcal{L}$ bằng numerical gradient; đường đồng mức giải thích đường đi zigzag. Chương 2 bổ sung cách scale dữ liệu lớn và tăng tốc hội tụ.

1.6 Tài liệu tham khảo

Tài liệu đọc thêm

1. Sebastian Ruder, *An overview of gradient descent optimization algorithms* — <https://ruder.io/optimizing-gradient-descent/>
2. *An Interactive Tutorial on Numerical Optimization* — <https://www.benfrederickson.com/numerical-optimization/>
3. Andrew Ng, *Gradient Descent* (Coursera Machine Learning) — <https://www.coursera.org/learn/machine-learning>

2 Chương 2. Biến thể và kỹ thuật nâng cao

Mở đầu chương

Tốc độ hội tụ của các thuật toán Gradient Descent khác nhau có thể so sánh qua tổng quan trong survey *An overview of gradient descent optimization algorithms*.

Chương 1 đã giới thiệu thuật toán Gradient Descent. Nghiệm cuối cùng của Gradient Descent phụ thuộc rất nhiều vào điểm khởi tạo và learning rate. Trong phần này, ta đề cập một vài phương pháp thường được dùng để khắc phục những hạn chế của GD. Đồng thời, các thuật toán biến thể của GD thường được áp dụng trong các mô hình Deep Learning cũng được tổng hợp.

2.1 1. Các thuật toán tối ưu Gradient Descent

2.1.1 1.1. Momentum

Nhắc lại thuật toán Gradient Descent.

Ôn tập nhanh

Để giải bài toán tìm điểm global optimal của hàm mất mát $J(\theta)$ (hàm mất mát cũng thường được ký hiệu là $J()$ với θ là tập hợp các tham số của mô hình), ta nhắc lại thuật toán GD.

Thuật toán Gradient Descent

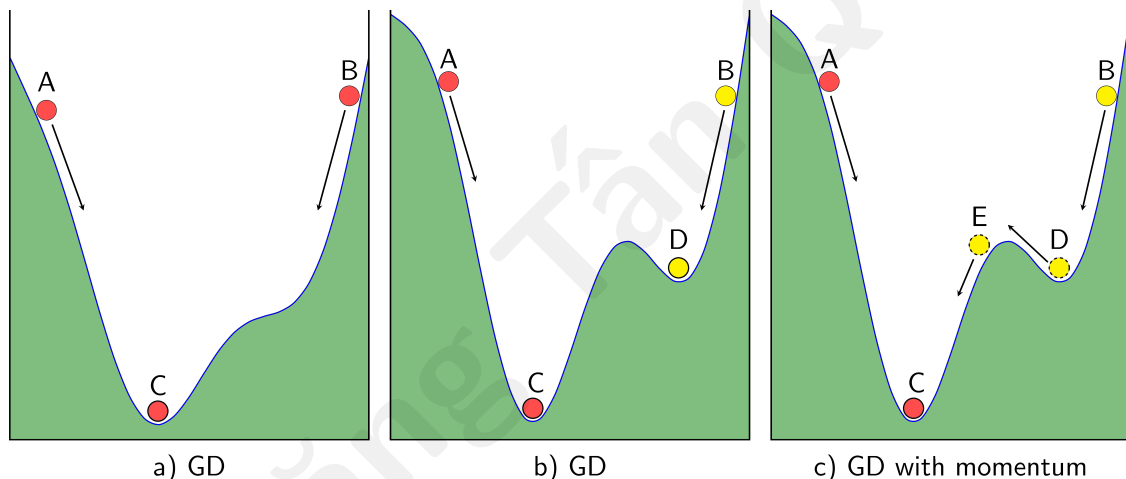
1. Dự đoán một điểm khởi tạo $\theta = \theta_0$.
2. Cập nhật θ đến khi đạt được kết quả chấp nhận được:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

với $\nabla_{\theta} J(\theta)$ là đạo hàm của hàm mất mát tại θ .

Gradient dưới góc nhìn vật lý.**Ví von vật lý**

Thuật toán GD thường được ví với tác dụng của trọng lực lên một hòn bi đặt trên một mặt có dạng như hình một thung lũng giống như Hình 1a) dưới đây. Bất kể ta đặt hòn bi ở A hay B thì cuối cùng hòn bi cũng sẽ lăn xuống và kết thúc ở vị trí C .

**Hai đáy thung lũng**

Tuy nhiên, nếu bề mặt có hai đáy thung lũng như Hình 1b) thì tùy vào việc đặt bi ở A hay B , vị trí cuối cùng của bi sẽ ở C hoặc D . Điểm D là một điểm local minimum chúng ta không mong muốn.

Vượt local minimum nhờ đà

Nếu suy nghĩ một cách vật lý hơn, vẫn trong Hình 1b), nếu vận tốc ban đầu của bi khi ở điểm B đủ lớn, khi bi lăn đến điểm D , theo đà, bi có thể tiếp tục di chuyển lên dốc phía bên trái của D . Và nếu giả sử vận tốc ban đầu lớn hơn nữa, bi có thể vượt dốc tới điểm E rồi lăn xuống C như trong Hình 1c). Đây chính là điều chúng ta mong muốn. Có thể đặt câu hỏi: liệu bi lăn từ A tới C có theo đà lăn tới E rồi tới D không? Điều này khó xảy ra hơn vì nếu so với dốc DE thì dốc CE cao hơn nhiều.

Dựa trên hiện tượng này, một thuật toán được ra đời nhằm khắc phục việc nghiệm của GD rơi vào một điểm local minimum không mong muốn. Thuật toán đó có tên là

Momentum (theo đà).

Gradient Descent với Momentum.

Biểu diễn momentum bằng toán học

Để biểu diễn momentum bằng toán học thì chúng ta phải làm thế nào?

Trong GD, chúng ta cần tính lượng thay đổi ở thời điểm t để cập nhật vị trí mới cho nghiệm (tức hòn bi). Nếu chúng ta coi đại lượng này như vận tốc v_t trong vật lý, vị trí mới của hòn bi sẽ là $\theta_{t+1} = \theta_t - v_t$. Dấu trừ thể hiện việc phải di chuyển ngược với đạo hàm. Công việc của chúng ta bây giờ là tính đại lượng v_t sao cho nó vừa mang thông tin của độ dốc (tức đạo hàm), vừa mang thông tin của đà, tức vận tốc trước đó v_{t-1} (chúng ta coi như vận tốc ban đầu $v_0 = 0$). Một cách đơn giản nhất, ta có thể cộng (có trọng số) hai đại lượng này lại:

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$

Trong đó γ thường được chọn là một giá trị khoảng 0.9, v_t là vận tốc tại thời điểm trước đó, $\nabla_{\theta} J(\theta)$ chính là độ dốc của điểm trước đó. Sau đó vị trí mới của hòn bi được xác định như sau:

$$\theta = \theta - v_t$$

Không gian nhiều chiều

Thuật toán đơn giản này tỏ ra rất hiệu quả trong các bài toán thực tế (trong không gian nhiều chiều, cách tính toán cũng hoàn toàn tương tự). Dưới đây là một ví dụ trong không gian một chiều.

Một ví dụ nhỏ. Chúng ta xem xét một hàm đơn giản có hai điểm local minimum, trong đó một điểm là global minimum:

$$f(x) = x^2 + 10 \sin(x)$$

Có đạo hàm là: $f'(x) = 2x + 10 \cos(x)$.

So sánh GD và GD với Momentum

Hình 2 dưới đây thể hiện sự khác nhau giữa thuật toán GD và thuật toán GD với Momentum.

Thiếu hình minh họa

Không tìm thấy images/02_gradient_descent_part2/img04.png. Chạy download_mlcb_images.py.

Thiếu hình minh họa

Không tìm thấy images/02_gradient_descent_part2/img05.png. Chạy download_mlcb_images.py.

Hình bên trái là đường đi của nghiệm khi **không** sử dụng Momentum: thuật toán hội tụ sau chỉ 5 vòng lặp nhưng nghiệm tìm được là nghiệm local minimum.

Hình bên phải là đường đi của nghiệm khi **có** sử dụng Momentum: hòn bi đã có thể vượt dốc tới khu vực gần điểm global minimum, sau đó dao động xung quanh điểm này, giảm tốc rồi cuối cùng tới đích. Mặc dù mất nhiều vòng lặp hơn, GD với Momentum cho chúng ta nghiệm chính xác hơn. Quan sát đường đi của hòn bi trong trường hợp này, chúng ta thấy rằng điều này giống với vật lý hơn!

Hàm GD_momentum trong Python

Nếu biết trước điểm đặt bi ban đầu **theta**, đạo hàm của hàm mất mát tại một điểm bất kỳ **grad(theta)**, lượng thông tin lưu trữ từ vận tốc trước đó **gamma** và learning rate **eta**, chúng ta có thể viết hàm số GD_momentum trong Python như sau:

```
# check convergence
def has_converged(theta_new, grad):
    return np.linalg.norm(grad(theta_new))/
    len(theta_new) < 1e-3

def GD_momentum(theta_init, grad, eta, gamma):
    # Suppose we want to store history of theta
    theta = [theta_init]
    v_old = np.zeros_like(theta_init)
    for it in range(100):
        v_new = gamma*v_old + eta*grad(theta[-1])
        theta_new = theta[-1] - v_new
        if has_converged(theta_new, grad):
            break
        theta.append(theta_new)
        v_old = v_new
    return theta
    # this variable includes all points in the path
    # if you just want the final answer,
    # use `return theta[-1]`
```

2.1.2 1.2. Nesterov accelerated gradient (NAG)

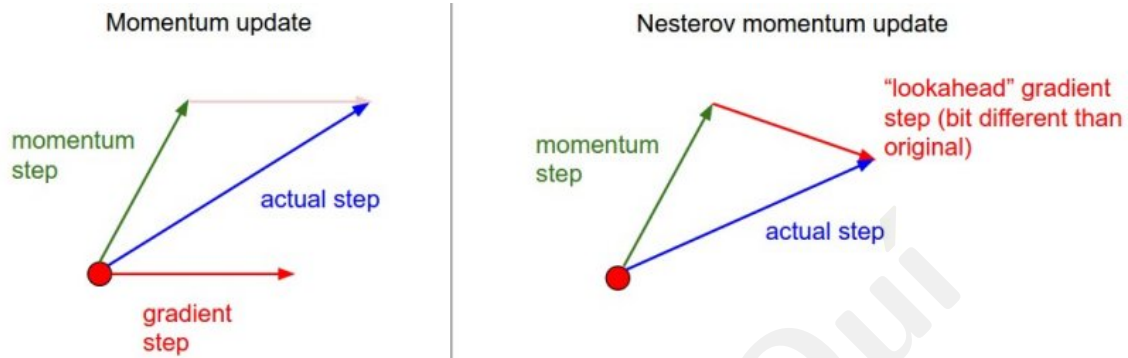
Hạn chế của Momentum thuần

Momentum giúp hòn bi vượt qua được dốc local minimum, tuy nhiên, có một hạn chế chúng ta có thể thấy trong ví dụ trên: khi tới gần đích, momentum vẫn mất khá nhiều thời gian trước khi dừng lại. Lý do lại cũng chính là vì có đà. Có một phương pháp khác tiếp tục giúp khắc phục điều này, phương pháp đó mang tên **Nesterov accelerated gradient (NAG)**, giúp cho thuật toán hội tụ nhanh hơn.

Ý tưởng chính.

Nhìn trước một bước

Ý tưởng cơ bản là dự đoán hướng đi trong tương lai, tức nhìn trước một bước! Cụ thể, nếu sử dụng số hạng momentum γv_{t-1} để cập nhật thì ta có thể xấp xỉ được vị trí tiếp theo của hòn bi là $\theta - \gamma v_{t-1}$ (chúng ta không đính kèm phần gradient ở đây vì sẽ sử dụng nó trong bước cuối cùng). Vậy, thay vì sử dụng gradient của điểm hiện tại, NAG đi trước một bước, sử dụng gradient của điểm tiếp theo. Theo dõi hình dưới đây:



- Với momentum thông thường: lượng thay đổi là tổng của hai vector: momentum vector và gradient ở thời điểm hiện tại.
- Với Nesterov momentum: lượng thay đổi là tổng của hai vector: momentum vector và gradient ở thời điểm được xấp xỉ là điểm tiếp theo.

Công thức cập nhật. Công thức cập nhật của NAG được cho như sau:

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1}) \quad (1)$$

$$\theta = \theta - v_t \quad (2)$$

Điểm tính đạo hàm

Để ý một chút: điểm được tính đạo hàm đã thay đổi so với Momentum thông thường.

Ví dụ minh họa.**So sánh Momentum và NAG (Linear Regression)**

Dưới đây là ví dụ so sánh Momentum và NAG cho bài toán Linear Regression:

Thiếu hình minh họa

Không tìm thấy images/02_gradient_descent_part2/img07.png. Chạy download_mlc_b_images.py.

Thiếu hình minh họa

Không tìm thấy images/02_gradient_descent_part2/img08.png. Chạy download_mlcb_images.py.

Hình bên trái là đường đi của nghiệm với phương pháp Momentum: nghiệm đi khá zigzag và mất nhiều vòng lặp hơn. Hình bên phải là đường đi của nghiệm với phương pháp NAG: nghiệm hội tụ nhanh hơn, và đường đi ít zigzag hơn.

2.1.3 1.3. Các thuật toán khác**Adagrad, Adam, RMSprop — tổng quan**

Ngoài Momentum và NAG, trong Deep Learning thường dùng các optimizer **thích ứng learning rate theo từng tọa độ**: **Adagrad**, **RMSprop**, **Adam** (kết hợp momentum và RMSprop). Ý tưởng chi tiết và bảng so sánh xem mục **Tổng hợp và củng cố** cuối giáo trình; tài liệu mở rộng: Ruder, *An overview of gradient descent optimization algorithms*.

Thực hành

Trong PyTorch/TensorFlow, tham số optimizer (SGD + momentum, Adam, ...) triển khai các công thức này; người học nên map từng symbol trong slide sang cập nhật θ ở trên.

2.2 2. Biến thể của Gradient Descent**Ví dụ Linear Regression**

Tiếp tục dùng bài toán Linear Regression làm ví dụ. Hàm mất mát và đạo hàm của nó cho bài toán này lần lượt là (để cho thuận tiện, trong bài này ta dùng ký hiệu $\mathbf{X}$ thay cho dữ liệu mở rộng $\tilde{\mathbf{X}}$):

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{2N} \sum_{i=1}^N (\mathbf{x}_i \mathbf{w} - y_i)^2$$

và:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i^T (\mathbf{x}_i \mathbf{w} - y_i)$$

2.2.1 2.1. Batch Gradient Descent

Batch Gradient Descent

Thuật toán Gradient Descent chúng ta nói từ đầu phần 1 đến giờ còn được gọi là **Batch Gradient Descent**. Batch ở đây được hiểu là tất cả, tức khi cập nhật $\theta = \mathbf{w}$, chúng ta sử dụng tất cả các điểm dữ liệu $\mathbf{x}_i$.

Hạn chế với dữ liệu lớn và online learning

Cách làm này có một vài hạn chế đối với cơ sở dữ liệu có vô cùng nhiều điểm (hơn 1 tỉ người dùng của Facebook chẳng hạn). Việc phải tính toán lại đạo hàm với tất cả các điểm này sau mỗi vòng lặp trở nên cồng kềnh và không hiệu quả. Thêm nữa, thuật toán này được coi là không hiệu quả với **online learning**.

Online learning là khi cơ sở dữ liệu được cập nhật liên tục (thêm người dùng đăng ký hàng ngày chẳng hạn), mỗi lần thêm vài điểm dữ liệu mới. Kéo theo đó là mô hình của chúng ta cũng phải thay đổi một chút để phù hợp với các dữ liệu mới này. Nếu làm theo Batch Gradient Descent, tức tính lại đạo hàm của hàm mất mát tại tất cả các điểm dữ liệu, thì thời gian tính toán sẽ rất lâu, và thuật toán của chúng ta coi như không online nữa do mất quá nhiều thời gian tính toán.

Trên thực tế, có một thuật toán đơn giản hơn và tỏ ra rất hiệu quả, có tên gọi là **Stochastic Gradient Descent (SGD)**.

2.2.2 2.2. Stochastic Gradient Descent

Stochastic Gradient Descent (SGD)

Trong thuật toán này, tại một thời điểm, ta chỉ tính đạo hàm của hàm mất mát dựa trên **chỉ một** điểm dữ liệu $\mathbf{x}_i$ rồi cập nhật θ dựa trên đạo hàm này. Việc này được thực hiện với từng điểm trên toàn bộ dữ liệu, sau đó lặp lại quá trình trên. Thuật toán rất đơn giản này trên thực tế lại làm việc rất hiệu quả.

Epoch và ưu điểm SGD

Mỗi lần duyệt một lượt qua tất cả các điểm trên toàn bộ dữ liệu được gọi là một **epoch**. Với GD thông thường thì mỗi epoch ứng với 1 lần cập nhật θ ; với SGD thì mỗi epoch ứng với N lần cập nhật θ với N là số điểm dữ liệu. Nhìn vào một mặt, việc cập nhật từng điểm một như thế này có thể làm giảm đi tốc độ thực hiện 1 epoch. Nhưng nhìn vào một mặt khác, SGD chỉ yêu cầu một lượng epoch rất nhỏ (thường là 10 cho lần đầu tiên, sau đó khi có dữ liệu mới thì chỉ cần chạy dưới một epoch là đã có nghiệm tốt). Vì vậy SGD phù hợp với các bài toán có lượng cơ sở dữ liệu lớn (chủ yếu là Deep Learning) và các bài toán yêu cầu mô hình thay đổi liên tục, tức online learning.

Thứ tự lựa chọn điểm dữ liệu.

Shuffle sau mỗi epoch

Một điểm cần lưu ý: sau mỗi epoch, chúng ta cần **shuffle** (xáo trộn) thứ tự của các dữ liệu để đảm bảo tính ngẫu nhiên. Việc này cũng ảnh hưởng tới hiệu năng của SGD.

Một cách toán học, quy tắc cập nhật của SGD là:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; \mathbf{x}_i; \mathbf{y}_i)$$

trong đó $J(\theta; \mathbf{x}_i; \mathbf{y}_i)$ là hàm mất mát với chỉ 1 cặp điểm dữ liệu (input, label) là $(\mathbf{x}_i, \mathbf{y}_i)$.

Kết hợp với Momentum, AdaGrad, ...

Chúng ta hoàn toàn có thể áp dụng các thuật toán tăng tốc GD như Momentum, AdaGrad, ... vào SGD.

Ví dụ với bài toán Linear Regression. Với bài toán Linear Regression, $\theta = \mathbf{w}$, hàm mất mát tại một điểm dữ liệu là:

$$J(\mathbf{w}; \mathbf{x}_i; y_i) = \frac{1}{2}(\mathbf{x}_i \mathbf{w} - y_i)^2$$

Đạo hàm theo $\mathbf{w}$ tương ứng là:

$$\nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i; y_i) = \mathbf{x}_i^T (\mathbf{x}_i \mathbf{w} - y_i)$$

Hàm sgrad và SGD trong Python

Và dưới đây là hàm số trong Python để giải Linear Regression theo SGD:

```
# single point gradient
def sgrad(w, i, rd_id):
    true_i = rd_id[i]
    xi = Xbar[true_i, :]
    yi = y[true_i]
    a = np.dot(xi, w) - yi
    return (xi*a).reshape(2, 1)

def SGD(w_init, grad, eta):
    w = [w_init]
    w_last_check = w_init
    iter_check_w = 10
    N = X.shape[0]
    count = 0
    for it in range(10):
        # shuffle data
        rd_id = np.random.permutation(N)
        for i in range(N):
            count += 1
            g = sgrad(w[-1], i, rd_id)
            w_new = w[-1] - eta*g
            w.append(w_new)
```

```

if count%iter_check_w == 0:
    w_this_check = w_new
    if np.linalg.norm(w_this_check - w_last_check)/len(w_init) < 1e-3:
        return w
    w_last_check = w_this_check
return w

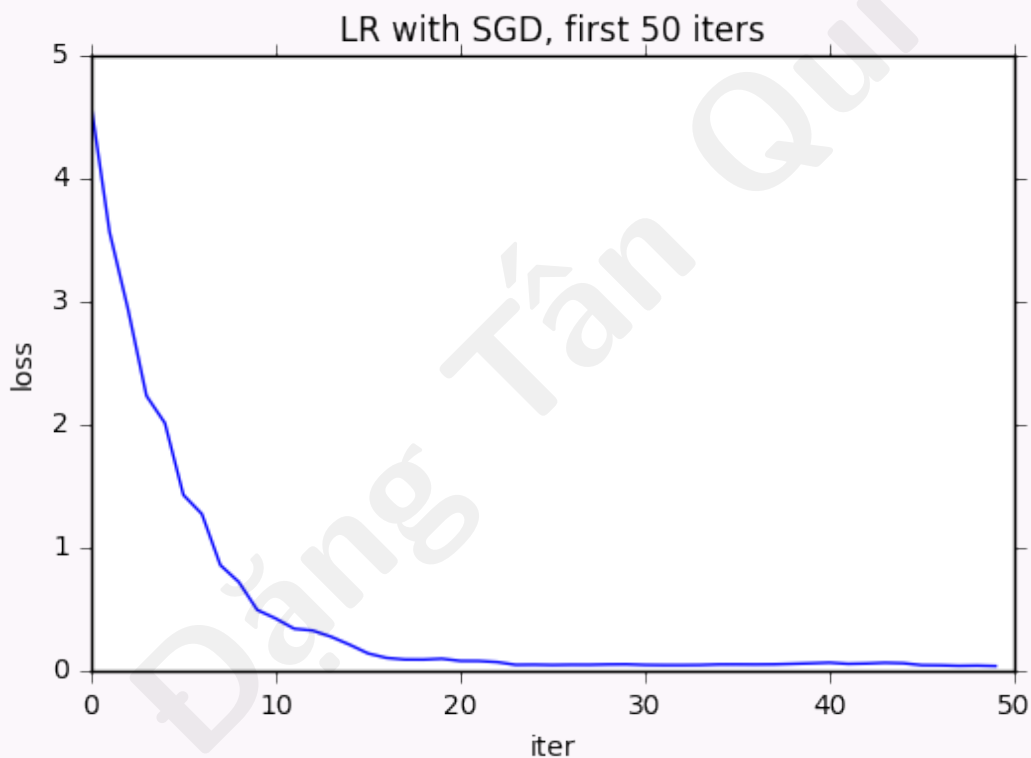
```

Kết quả SGD trên Linear Regression

Kết quả được cho như hình dưới đây (với dữ liệu được tạo giống như ở phần 1).

Thiếu hình minh họa

Không tìm thấy images/02_gradient_descent_part2/img09.png. Chạy download_mlcb_images.py.



Trái: đường đi của nghiệm với SGD. **Phải:** giá trị của loss function tại 50 vòng lặp đầu tiên.

Hình bên trái mô tả đường đi của nghiệm. Chúng ta thấy rằng đường đi khá zigzag chứ không mượt như khi sử dụng GD. Điều này là dễ hiểu vì một điểm dữ liệu không thể đại diện cho toàn bộ dữ liệu được. Tuy nhiên, chúng ta cũng thấy rằng thuật toán hội tụ khá nhanh đến vùng lân cận của nghiệm. Với 1000 điểm dữ liệu, SGD chỉ cần gần 3 epoches (2911 tương ứng với 2911 lần cập nhật, mỗi lần lấy 1 điểm). Nếu so với con số 49 vòng lặp (epoches) như kết quả tốt nhất có được bằng GD, thì kết quả này lợi hơn rất nhiều. Hình bên phải mô tả hàm mất mát cho toàn bộ dữ liệu sau khi chỉ sử dụng 50 điểm dữ liệu đầu tiên. Mặc dù không mượt, tốc độ hội tụ vẫn rất nhanh.

Thực tế cho thấy chỉ lấy khoảng 10 điểm là ta đã có thể xác định được gần đúng phương trình đường thẳng cần tìm rồi. Đây chính là ưu điểm của SGD — hội tụ rất nhanh.

2.2.3 2.3. Mini-batch Gradient Descent

Mini-batch Gradient Descent

Khác với SGD, mini-batch sử dụng một số lượng n lớn hơn 1 (nhưng vẫn nhỏ hơn tổng số dữ liệu N rất nhiều). Giống với SGD, Mini-batch Gradient Descent bắt đầu mỗi epoch bằng việc xáo trộn ngẫu nhiên dữ liệu rồi chia toàn bộ dữ liệu thành các mini-batch, mỗi mini-batch có n điểm dữ liệu (trừ mini-batch cuối có thể có ít hơn nếu N không chia hết cho n). Mỗi lần cập nhật, thuật toán này lấy ra một mini-batch để tính toán đạo hàm rồi cập nhật.

Công thức có thể viết dưới dạng:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; \mathbf{x}_{i:i+n}; \mathbf{y}_{i:i+n})$$

Với $\mathbf{x}_{i:i+n}$ được hiểu là dữ liệu từ thứ i tới thứ $i + n - 1$ (theo ký hiệu của Python). Dữ liệu này sau mỗi epoch là khác nhau vì chúng cần được xáo trộn.

Kết hợp Momentum, Adagrad, Adadelta, ...

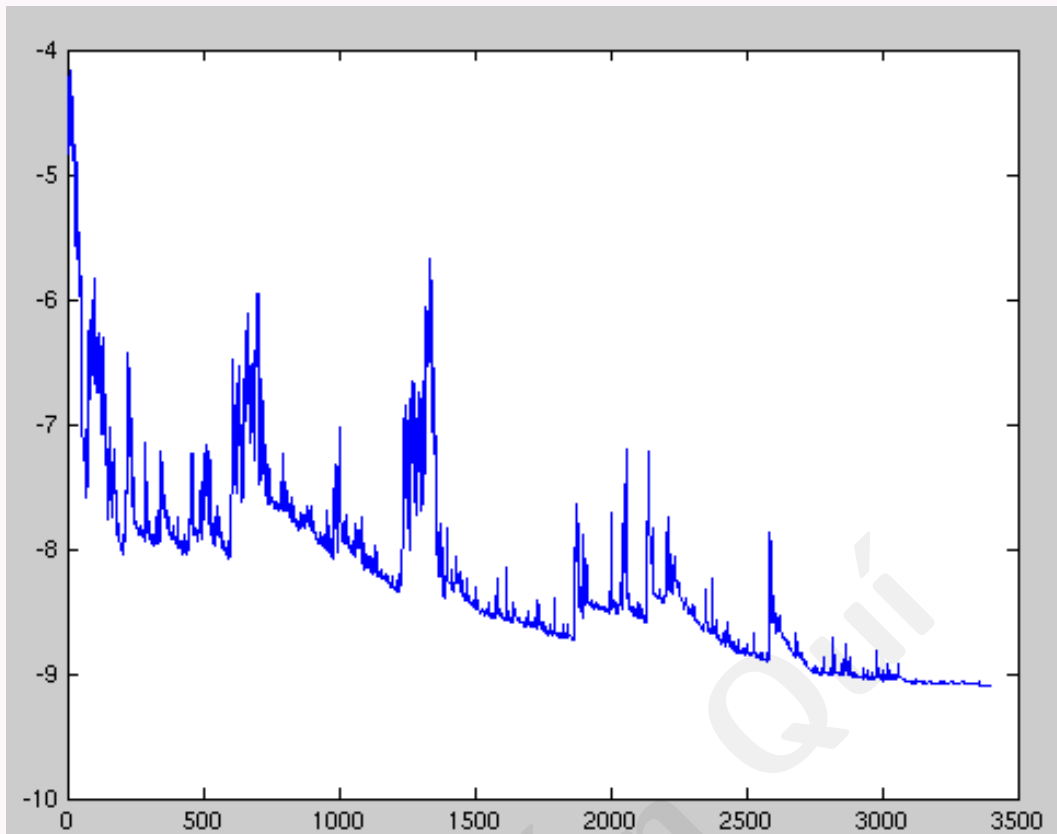
Một lần nữa, các thuật toán khác cho GD như Momentum, Adagrad, Adadelta, ... cũng có thể được áp dụng vào đây.

Ứng dụng thực tế

Mini-batch GD được sử dụng trong hầu hết các thuật toán Machine Learning, đặc biệt là trong Deep Learning. Giá trị n thường được chọn là khoảng từ 50 đến 100.

Dao động hàm mất mát (fluctuate)

Dưới đây là ví dụ về giá trị của hàm mất mát mỗi khi cập nhật tham số θ của một bài toán khác phức tạp hơn.



Hàm mất mát nhảy lên nhảy xuống (fluctuate) sau mỗi lần cập nhật nhưng nhìn chung giảm dần và có xu hướng hội tụ về cuối.

Để có thêm thông tin chi tiết hơn, có thể tham khảo mục Stochastic Gradient Descent trên Wikipedia.

2.3 3. Stopping Criteria (điều kiện dừng)

Khi nào dừng?

Có một điểm cũng quan trọng mà từ đầu chưa nhắc đến: khi nào thì chúng ta biết thuật toán đã hội tụ và dừng lại?

Trong thực nghiệm, có một vài phương pháp như dưới đây:

1. **Giới hạn số vòng lặp:** đây là phương pháp phổ biến nhất và cũng để đảm bảo rằng chương trình chạy không quá lâu. Tuy nhiên, một nhược điểm của cách làm này là có thể thuật toán dừng lại trước khi đủ gần với nghiệm.
2. **So sánh gradient của nghiệm tại hai lần cập nhật liên tiếp:** khi nào giá trị này đủ nhỏ thì dừng lại. Phương pháp này cũng có một nhược điểm lớn là việc tính đạo hàm đôi khi trở nên quá phức tạp (ví dụ như khi có quá nhiều dữ liệu); nếu áp dụng phương pháp này thì coi như ta không được lợi khi sử dụng SGD và mini-batch GD.
3. **So sánh giá trị của hàm mất mát của nghiệm tại hai lần cập nhật liên tiếp,** khi nào giá trị này đủ nhỏ thì dừng lại. Nhược điểm của phương pháp này là nếu tại một thời điểm, đồ thị hàm số có dạng bằng phẳng tại một khu vực nhưng khu vực đó không chứa

điểm local minimum (khu vực này thường được gọi là **saddle points**), thuật toán cũng dừng lại trước khi đạt giá trị mong muốn.

4. Trong **SGD và mini-batch GD**, cách thường dùng là so sánh nghiệm sau một vài lần cập nhật. Trong đoạn code Python phía trên về SGD, ta áp dụng việc so sánh này mỗi khi nghiệm được cập nhật 10 lần. Việc làm này cũng tỏ ra khá hiệu quả.

2.4 4. Một phương pháp tối ưu đơn giản khác: Newton's method

First-order và second-order

Nhân tiện đang nói về tối ưu, ta xin giới thiệu một phương pháp nữa có cách giải thích đơn giản: **Newton's method**. Các phương pháp GD đã trình bày còn được gọi là **first-order methods**, vì lời giải tìm được dựa trên đạo hàm bậc nhất của hàm số. Newton's method là một **second-order method**, tức lời giải yêu cầu tính đến đạo hàm bậc hai. Nhắc lại rằng, cho tới thời điểm này, chúng ta luôn giải phương trình đạo hàm của hàm mất mát bằng 0 để tìm các điểm local minimum. (Và trong nhiều trường hợp, coi nghiệm tìm được là nghiệm của bài toán tìm giá trị nhỏ nhất của hàm mất mát.) Có một thuật toán nổi tiếng giúp giải bài toán $f(x) = 0$, thuật toán đó có tên là Newton's method.

2.4.1 Newton's method cho giải phương trình $f(x) = 0$

Ý tưởng hình học

Thuật toán Newton's method được mô tả trong hình động minh họa dưới đây:

Thiếu hình minh họa

Không tìm thấy images/02_gradient_descent_part2/img12.png. Chạy download_mlc_b_images.py.

Ý tưởng giải bài toán $f(x) = 0$ bằng phương pháp Newton's method như sau. Xuất phát từ một điểm x_0 được cho là gần với nghiệm x^* . Sau đó vẽ đường tiếp tuyến (mặt tiếp tuyến trong không gian nhiều chiều) với đồ thị hàm số $y = f(x)$ tại điểm trên đồ thị có hoành độ x_0 . Giao điểm x_1 của đường tiếp tuyến này với trục hoành được xem là gần với nghiệm x^* hơn. Thuật toán lặp lại với điểm mới x_1 và cứ như vậy đến khi ta được $f(x_t) \approx 0$.

Công thức lập trình

Đó là ý nghĩa hình học của Newton's method; chúng ta cần một công thức để có thể dựa vào đó để lập trình. Việc này không quá phức tạp sau khóa Giải tích I. Thật vậy, phương trình tiếp tuyến với đồ thị của hàm $f(x)$ tại điểm có hoành độ x_t là:

$$y = f'(x_t)(x - x_t) + f(x_t)$$

Giao điểm của đường thẳng này với trục x tìm được bằng cách giải phương trình vế phải của biểu thức trên bằng 0, tức là:

$$x = x_t - \frac{f(x_t)}{f'(x_t)} \triangleq x_{t+1}$$

2.4.2 Newton's method trong bài toán tìm local minimum

Áp dụng phương pháp này cho việc giải phương trình $f'(x) = 0$ ta có:

$$x_{t+1} = x_t - (f''(x_t))^{-1} f'(x_t)$$

Và trong không gian nhiều chiều với θ là biến:

$$\theta = \theta - \mathbf{H}(J(\theta))^{-1} \nabla_{\theta} J(\theta)$$

trong đó $\mathbf{H}(J(\theta))$ là đạo hàm bậc hai của hàm mất mát (còn gọi là **Hessian matrix**). Biểu thức này là một ma trận nếu θ là một vector. Và $\mathbf{H}(J(\theta))^{-1}$ chính là nghịch đảo của ma trận đó.

2.4.3 Hạn chế của Newton's method

1. **Điểm khởi tạo phải rất gần với nghiệm x^* .** Ý tưởng sâu xa hơn của Newton's method là dựa trên khai triển Taylor của hàm số $f(x)$ tới đạo hàm thứ nhất:

$$0 = f(x^*) \approx f(x_t) + f'(x_t)(x_t - x^*)$$

Từ đó suy ra: $x^* \approx x_t - \frac{f(x_t)}{f'(x_t)}$. Một điểm rất quan trọng: khai triển Taylor chỉ đúng nếu x_t rất gần với x^* ! Dưới đây là một ví dụ kinh điển trên Wikipedia về việc Newton's method cho một dãy số phân kỳ (divergence): nghiệm là một điểm gần -2 ; tiếp tuyến của đồ thị hàm số tại điểm có hoành độ bằng 0 cắt trục hoành tại 1, và ngược lại. Trong trường hợp này, Newton's method không bao giờ hội tụ.

2. **Đạo hàm ở mẫu số gần 0.** Trong việc giải phương trình $f(x) = 0$, chúng ta có đạo hàm ở mẫu số. Khi đạo hàm này gần với 0, ta sẽ được một đường thẳng song song hoặc gần song song với trục hoành. Ta sẽ hoặc không tìm được giao điểm, hoặc được một giao điểm ở vô cùng. Đặc biệt, khi nghiệm chính là điểm có đạo hàm bằng 0, thuật toán gần như sẽ không tìm được nghiệm!
3. **Chi phí Hessian trong không gian nhiều chiều.** Khi áp dụng Newton's method cho bài toán tối ưu trong không gian nhiều chiều, chúng ta cần tính nghịch đảo của Hessian matrix. Khi số chiều và số điểm dữ liệu lớn, đạo hàm bậc hai của hàm mất mát sẽ là một ma trận rất lớn, ảnh hưởng tới cả memory và tốc độ tính toán của hệ thống.

2.5 5. Kết luận

Tóm tắt Chương 2

Chương này bổ sung **Momentum**, **NAG**, phân loại **Batch / SGD / mini-batch**, **điều kiện dừng** và ý tưởng **Newton**. Đây là các công cụ thực tế khi huấn luyện mô hình quy mô lớn. Phần **Tổng hợp và củng cố** (cuối giáo trình) gom bảng so sánh, Adam/RMSprop và câu hỏi ôn tập.

2.6 6. Tài liệu tham khảo

1. Newton's method — Wikipedia: https://en.wikipedia.org/wiki/Newton%27s_method

2. An overview of gradient descent optimization algorithms: <http://sebastianruder.com/optimizing-gradient-descent/>
3. Stochastic Gradient Descent — Wikipedia: https://en.wikipedia.org/wiki/Stochastic_gradient_descent
4. Stochastic Gradient Descent — Andrew Ng (Coursera lecture notes)
5. Nesterov, Y. (1983). A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$. *Doklady ANSSSR* (translated as *Soviet Math. Dokl.*), vol. 269, pp. 543–547.

Tổng hợp và củng cố

2.7 Bảng so sánh các biến thể GD

Phân loại theo dữ liệu và cập nhật

Phương pháp	Gradient dùng mỗi bước	Đặc điểm chính
Batch GD	Toàn bộ N mẫu	Ổn định; tốn kém khi N lớn
SGD	Một mẫu $\mathbf{x}_i$	Nhanh/epoch; nhiễu; cần shuffle
Mini-batch	n mẫu ($n \ll N$)	Chuẩn trong Deep Learning
Momentum / NAG	(kết hợp với trên)	Vượt vùng dốc thoải; NAG ít zigzag hơn
Newton	Hessian $\mathbf{H}^{-1} \nabla J$	Ít bước nếu gần nghiệm; đắt tính $\mathbf{H}$

2.8 Bổ trợ: các optimizer phổ biến trong Deep Learning

AdaGrad, RMSprop, Adam — ý tưởng ngắn gọn

Phần 2 chỉ liệt kê tên; dưới đây là khung hiểu để đọc tài liệu chuyên sâu (ví dụ Ruder, 2016).

- **AdaGrad**: chia learning rate cho căn bậc hai **tổng tích lũy bình phương gradient** theo từng tọa độ — học nhanh chiều ít cập nhật, chậm chiều gradient lớn; nhược điểm: tốc độ học có thể “chết” khi tích lũy quá lớn.
- **RMSprop**: dùng **trung bình trượt** bình phương gradient (EMA) thay tổng vô hạn — khắc phục AdaGrad trên dữ liệu không dừng.
- **Adam**: kết hợp **momentum** (EMA của gradient) và **RMSprop** (EMA của g^2); thường là lựa chọn mặc định đầu tiên khi huấn luyện mạng sâu, vẫn cần tinh chỉnh η .

Hàm lồi (convex) và tại sao LR quan trọng

Với hàm lồi (ví dụ MSE của Linear Regression), mọi local minimum là **global minimum**. GD với η đủ nhỏ sẽ hội tụ (lý thuyết). Trên thực tế:

- η quá nhỏ: hội tụ chậm, tốn epoch.

- η quá lớn: dao động hoặc **phân kỳ** (nhảy qua đáy).
- Đường đồng mức **đẹt/elongated** (ma trận $\bar{\mathbf{X}}^T \bar{\mathbf{X}}$ xấu điều kiện): GD zigzag — động lực cho Momentum, preconditioning, Adam.

Saddle point và non-convex

Hàm mất mát mạng sâu thường **không lồi**: có nhiều cực tiểu cục bộ, **saddle point** (gradient = 0 nhưng không phải cực tiểu). GD vẫn dùng rộng rãi vì đơn giản và kết hợp tốt với mini-batch + momentum; không đảm bảo global optimum.

2.9 Câu hỏi củng cố

Tự kiểm tra

1. Vì sao cập nhật $x_{t+1} = x_t - \eta f'(x_t)$ lại **giảm** f khi $|\eta f'(x_t)|$ đủ nhỏ và f trơn?
2. Giải thích sự khác nhau giữa **một epoch** của Batch GD và của SGD (số lần cập nhật θ).
3. Khi nào nên dùng **numerical gradient** thay vì gradient công thức?
4. Momentum giúp gì khi loss có hai “thung lũng” cục bộ?
5. Newton method cần thêm thông tin gì so với GD? Vì sao ít dùng trực tiếp cho mạng lớn?

Tóm tắt chương

Gradient Descent là **xương sống** của huấn luyện có gradient: đi ngược hướng tăng nhanh nhất của loss, lặp với bước η . Thực hành ML dùng **mini-batch SGD** kèm **momentum/NAG/Adam**, shuffle dữ liệu, và điều kiện dừng phù hợp. Hiểu rõ **gradient**, **learning rate** và **hình dạng đường đồng mức** là chìa khóa để debug mô hình không hội tụ.